

PARTE 2

¡Entendido, Sebastián! Ya que tu tecnología asignada es **KMP + Compose**, vamos a construir este Mock-CRUD directamente sobre el proyecto **TallerKMP** que configuramos en la clase anterior.

A este nivel de la carrera de Ingeniería de Sistemas, entender el árbol de renderizado a bajo nivel es lo que marca la diferencia en el diseño de software. Analizar cómo los componentes gráficos se comunican con el sistema operativo es una habilidad técnica rigurosa que sin duda elevará la calidad de tus análisis y reportes técnicos en el Laboratorio ADN Software.

Para cumplir exactamente con la rúbrica (Material 3, cero librerías externas, y el puente nativo para el Toast), vamos a usar el mismo poder del patrón **expect/actual** que aprendiste antes, pero esta vez para invocar hardware y contexto nativo de Android.

Aquí tienes el manual definitivo, paso a paso, para tu Taller de Clase 04.

Manual Práctico: CRUD Visual y Renderizado en KMP

Contexto del Framework (KMP + Compose)

A diferencia de React Native o NativeScript que instancian componentes de Android (**android.widget.TextView**), Compose dibuja **cada píxel directamente sobre un Canvas** (lienzo). Por lo tanto, tu interfaz será extremadamente rápida y fluida, pero requerirá que crucemos un "puente" cuando queramos usar un servicio antiguo del sistema, como un **Toast**.

Paso 1: Configurar el Reto Nativo (El Toast)

El profesor pide explícitamente usar **LocalContext.current** para el Toast. Como el código de nuestra interfaz vive en **commonMain** (que no sabe qué es un "Context" de Android), crearemos un componente puente.

1.1. En **commonMain** (La Promesa)

1. Ve a **composeApp > src > commonMain > kotlin > org.example.project**.
2. Crea un archivo llamado **NativePlatform.kt**.
3. Declara la promesa de un Toast nativo:

```
package org.example.project

import androidx.compose.runtime.Composable

// Prometemos un componente que lanzará un mensaje nativo
@Composable
```

```
expect fun NativeToast(message: String, showToast: Boolean, onDismiss: () -> Unit)
```

1.2. En `androidMain` (La Realidad en Android usando `LocalContext`)

1. Ve a `composeApp > src > androidMain > kotlin > org.example.project`.
2. Crea un archivo llamado `NativePlatform.android.kt`.
3. Cumplimos el requerimiento del profesor accediendo al contexto de Android:

```
package org.example.project

import android.widget.Toast
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

@Composable
actual fun NativeToast(message: String, showToast: Boolean, onDismiss: () -> Unit)
{
    if (showToast) {
        // REQUERIMIENTO CUMPLIDO: Uso de LocalContext.current
        val context = LocalContext.current
        Toast.makeText(context, message, Toast.LENGTH_SHORT).show()
        onDismiss() // Reseteamos el estado
    }
}
```

1.3. En `desktopMain` o `jvmMain` (Para que la PC no explote)

1. Ve a la carpeta de PC y crea `NativePlatform.desktop.kt`.

```
package org.example.project

import androidx.compose.runtime.Composable

@Composable
actual fun NativeToast(message: String, showToast: Boolean, onDismiss: () -> Unit)
{
    if (showToast) {
        println("🖥️ TOAST EN PC: $message") // Simulamos el Toast en consola
        onDismiss()
    }
}
```

Paso 2: Modelo de Datos y Estado de Navegación

Como no podemos usar librerías externas (ni de navegación ni de bases de datos), haremos todo con estados simples en memoria.

1. En `commonMain > kotlin > org.example.project`, abre tu archivo `App.kt`.
2. Al inicio del archivo (fuera de cualquier función), agrega nuestro modelo de datos y las pantallas:

```
// Datos quemados (Hardcoded)
data class Contact(val id: Int, val name: String, val role: String)

val dummyData = mutableListOf(
    Contact(1, "Alan Turing", "Matemático"),
    Contact(2, "Ada Lovelace", "Programadora"),
    Contact(3, "Linus Torvalds", "Ingeniero de Software")
)

// Gestor de Navegación simple
enum class ScreenType { LIST, FORM }
```

Paso 3: Dibujar la Interfaz (Cumpliendo Material 3)

Ahora, vamos a reemplazar la función `@Composable fun App()` que tenías, con las pantallas que pide el taller. Pega todo este código debajo del bloque anterior en `App.kt`.

3.1 La Lógica Principal

```
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Person
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun App() {
    var currentScreen by remember { mutableStateOf(ScreenType.LIST) }
    var showToast by remember { mutableStateOf(false) }

    // El puente nativo que creamos en el Paso 1
    NativeToast(
        message = "Eliminado con éxito",
        showToast = showToast,
        onDismiss = { showToast = false }
    )
}
```

```

    )

    MaterialTheme { // Aplica estilos Material 3 por defecto
        Surface(modifier = Modifier.fillMaxSize()) {
            when (currentScreen) {
                ScreenType.LIST -> ContactListScreen(
                    onNavigateToForm = { currentScreen = ScreenType.FORM },
                    onDeleteSuccess = { showToast = true }
                )
                ScreenType.FORM -> ContactFormScreen(
                    onNavigateBack = { currentScreen = ScreenType.LIST }
                )
            }
        }
    }
}

```

3.2 Pantalla de Lista (Read y Dialog de Delete)

Esta pantalla implementa la lista, los bordes redondeados y el Dialog nativo de Compose. Pégalo a continuación:

```

@Composable
fun ContactListScreen(onNavigateToForm: () -> Unit, onDeleteSuccess: () -> Unit) {
    var contactToDelete by remember { mutableStateOf<Contact?>(null) }

    Scaffold(
        floatingActionButton = {
            FloatingActionButton(onClick = onNavigateToForm) {
                Icon(Icons.Default.Add, contentDescription = "Crear")
            }
        }
    ) { padding ->
        LazyColumn(contentPadding = padding, modifier =
Modifier.fillMaxSize().padding(16.dp)) {
            items(dummyData) { contact ->
                // ElevatedCard cumple con las guías de elevación y bordes
                // redondeados de M3
                ElevatedCard(
                    modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp),
                    onClick = onNavigateToForm // Simula editar
                ) {
                    Row(
                        modifier = Modifier.padding(16.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(Icons.Default.Person, contentDescription = null,
modifier = Modifier.size(40.dp))
                        Spacer(modifier = Modifier.width(16.dp))
                        Column(modifier = Modifier.weight(1f)) {

```

```

                Text(contact.name, style =
MaterialTheme.typography.titleMedium)
                Text(contact.role, style =
MaterialTheme.typography.bodyMedium)
            }
            IconButton(onClick = { contactToDelete = contact }) {
                Icon(Icons.Default.Delete, contentDescription =
"Eliminar", tint = MaterialTheme.colorScheme.error)
            }
        }
    }
}

// Flujo de Eliminación (Dialog)
contactToDelete?.let { contact ->
    AlertDialog(
        onDismissRequest = { contactToDelete = null },
        title = { Text("Eliminar Registro") },
        text = { Text("¿Estás seguro de eliminar a ${contact.name}?") },
        confirmButton = {
            TextButton(onClick = {
                dummyData.remove(contact)
                contactToDelete = null
                onDeleteSuccess() // Lanza el Toast Nativo
            }) {
                Text("Sí, eliminar", color =
MaterialTheme.colorScheme.error)
            }
        },
        dismissButton = {
            TextButton(onClick = { contactToDelete = null }) {
                Text("Cancelar")
            }
        }
    )
}
}
}

```

3.3 Pantalla de Formulario (Create/Update)

Pega esto al final para cumplir con los inputs y selectores (switches).

```

@Composable
fun ContactFormScreen(onNavigateBack: () -> Unit) {
    var name by remember { mutableStateOf("") }
    var role by remember { mutableStateOf("") }
    var isActive by remember { mutableStateOf(true) }

    Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {

```

```
Text("Nuevo Registro", style = MaterialTheme.typography.headlineMedium)
Spacer(modifier = Modifier.height(16.dp))

OutlinedTextField(
    value = name,
    onChange = { name = it },
    label = { Text("Nombre Completo") },
    modifier = Modifier.fillMaxWidth()
)
Spacer(modifier = Modifier.height(8.dp))

OutlinedTextField(
    value = role,
    onChange = { role = it },
    label = { Text("Cargo") },
    modifier = Modifier.fillMaxWidth()
)
Spacer(modifier = Modifier.height(16.dp))

Row(verticalAlignment = Alignment.CenterVertically) {
    Text("Usuario Activo", modifier = Modifier.weight(1f))
    Switch(checked = isActive, onCheckedChange = { isActive = it })
}

Spacer(modifier = Modifier.weight(1f)) // Empuja los botones abajo

Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
    OutlinedButton(onClick = onNavigateBack) { Text("Cancelar") }
    Button(onClick = onNavigateBack) { Text("Guardar") }
}
}
```

¡Con gusto, Sebastián! Tienes toda la razón en pedir esta corrección. Ya que vimos exactamente cómo se muestra tu Layout Inspector en la vida real (con los cubitos azules de Compose), debemos ajustar ese paso en el manual para que coincida 100% con lo que vas a capturar y con el argumento exacto que vas a presentar.

Aquí tienes la **Fase Final corregida**, lista para que la reemplaces en tu documento:

Fase Final: Capturar el Entregable (Layout Inspector)

Aquí es donde demuestras académicamente por qué KMP es diferente a los frameworks híbridos.

1. **Ejecuta la app** en tu emulador de Android (asegúrate de seleccionar `composeApp`).
2. En Android Studio, ve a la barra superior y selecciona **Tools > Layout Inspector** (o busca el ícono de las capas superpuestas en la barra derecha).
3. Asegúrate de seleccionar tu emulador activo y el proceso de tu app en la parte superior izquierda de la ventana del Inspector.

4. **Analizando el Component Tree:** A la izquierda verás la jerarquía. Expande todo hasta que llegues al núcleo de tu lista.
5. **Lo que debes destacar en tu captura:** Al desplegar el árbol, notarás que todos los elementos (como `App`, `MaterialTheme`, `Scaffold`, `LazyColumn`, y `ElevatedCard`) tienen un ícono de un **cubito azul**. Esto significa que son "Nodos de Compose". **No existen** las clásicas etiquetas nativas de Android como `android.widget.TextView` o `android.widget.Button` por cada elemento que ves en la pantalla.
6. **Tu argumento en el reporte:** *"Como se evidencia en la captura del Layout Inspector, Kotlin Multiplatform y Jetpack Compose no instancian múltiples Widgets nativos por cada elemento visual (a diferencia de React Native o NativeScript). La jerarquía está compuesta exclusivamente por nodos declarativos puros. Todo cuelga de un solo contenedor raíz que actúa como un lienzo (Canvas) donde el motor gráfico dibuja los elementos directamente, logrando un rendimiento superior, y cruzando el puente nativo únicamente para servicios específicos del sistema operativo, como acceder al LocalContext para los Toasts."*